```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, regularizers
from sklearn.model_selection import train_test_split

tf.random.set_seed(42)
np.random.seed(42)

df=pd.read_csv("mnist_784_csv.csv")
x = df.iloc[:, :-1].values.astype("float32") / 255.0
print("Images:", x.shape)
print("Pixel range:", x.min(), "to", x.max())
```

```
Images: (70000, 784)
Pixel range: 0.0 to 1.0
```

```python
x_train, x_test = train_test_split(x, test_size=0.2, random_state=42)
print("Train:", x_train.shape, "| Test:", x_test.shape)
```

```
Train: (56000, 784) | Test: (14000, 784)
```

```python
ENCODING_DIM = 64
L1_STRENGTH = 1e-8

inp = keras.Input(shape=(784,))
encoded = layers.Dense(ENCODING_DIM, activation="relu",

activity_regularizer=regularizers.l1(L1_STRENGTH))(inp)
decoded = layers.Dense(784, activation="sigmoid")(encoded)

autoencoder = keras.Model(inp, decoded)
encoder = keras.Model(inp, encoded)

autoencoder.compile(optimizer="adam", loss="binary_crossentropy")
autoencoder.summary()
```

```
Model: "functional_6"
```

```
┏━━━
```

```
│ dense_2 (Dense)                 │ (None, 64)             │
50,240 │
├─────────────────────────────────┼────────────────────────┼──────────
──────┤
│ dense_3 (Dense)                 │ (None, 784)            │
50,960 │
└─────────────────────────────────┴────────────────────────┴──────────
──────┘

 Total params: 101,200 (395.31 KB)

 Trainable params: 101,200 (395.31 KB)

 Non-trainable params: 0 (0.00 B)
```

```
history = autoencoder.fit(
    x_train, x_train,
    validation_data=(x_test, x_test),
    epochs=30,
    batch_size=256,
)
```

```
Epoch 1/30
219/219 ━━━━━━━━━━━━━━━━━━━━ 5s 17ms/step - loss: 0.2538 - val_loss:
0.1687
Epoch 2/30
219/219 ━━━━━━━━━━━━━━━━━━━━ 2s 10ms/step - loss: 0.1479 - val_loss:
0.1320
Epoch 3/30
219/219 ━━━━━━━━━━━━━━━━━━━━ 2s 10ms/step - loss: 0.1220 - val_loss:
0.1134
Epoch 4/30
219/219 ━━━━━━━━━━━━━━━━━━━━ 3s 10ms/step - loss: 0.1074 - val_loss:
0.1021
Epoch 5/30
219/219 ━━━━━━━━━━━━━━━━━━━━ 2s 10ms/step - loss: 0.0981 - val_loss:
0.0945
Epoch 6/30
219/219 ━━━━━━━━━━━━━━━━━━━━ 4s 17ms/step - loss: 0.0915 - val_loss:
0.0890
Epoch 7/30
219/219 ━━━━━━━━━━━━━━━━━━━━ 2s 10ms/step - loss: 0.0869 - val_loss:
0.0850
Epoch 8/30
219/219 ━━━━━━━━━━━━━━━━━━━━ 2s 10ms/step - loss: 0.0835 - val_loss:
0.0822
Epoch 9/30
219/219 ━━━━━━━━━━━━━━━━━━━━ 2s 10ms/step - loss: 0.0812 - val_loss:
0.0802
Epoch 10/30
```

```
219/219 ━━━━━━━━━━━━━━━━━━━━ 3s 10ms/step - loss: 0.0795 - val_loss:
0.0789
Epoch 11/30
219/219 ━━━━━━━━━━━━━━━━━━━━ 3s 15ms/step - loss: 0.0783 - val_loss:
0.0778
Epoch 12/30
219/219 ━━━━━━━━━━━━━━━━━━━━ 3s 12ms/step - loss: 0.0774 - val_loss:
0.0770
Epoch 13/30
219/219 ━━━━━━━━━━━━━━━━━━━━ 2s 10ms/step - loss: 0.0767 - val_loss:
0.0765
Epoch 14/30
219/219 ━━━━━━━━━━━━━━━━━━━━ 2s 10ms/step - loss: 0.0762 - val_loss:
0.0760
Epoch 15/30
219/219 ━━━━━━━━━━━━━━━━━━━━ 2s 10ms/step - loss: 0.0758 - val_loss:
0.0756
Epoch 16/30
219/219 ━━━━━━━━━━━━━━━━━━━━ 3s 13ms/step - loss: 0.0755 - val_loss:
0.0754
Epoch 17/30
219/219 ━━━━━━━━━━━━━━━━━━━━ 4s 10ms/step - loss: 0.0752 - val_loss:
0.0751
Epoch 18/30
219/219 ━━━━━━━━━━━━━━━━━━━━ 2s 10ms/step - loss: 0.0750 - val_loss:
0.0750
Epoch 19/30
219/219 ━━━━━━━━━━━━━━━━━━━━ 2s 10ms/step - loss: 0.0749 - val_loss:
0.0748
Epoch 20/30
219/219 ━━━━━━━━━━━━━━━━━━━━ 2s 10ms/step - loss: 0.0747 - val_loss:
0.0747
Epoch 21/30
219/219 ━━━━━━━━━━━━━━━━━━━━ 4s 18ms/step - loss: 0.0746 - val_loss:
0.0746
Epoch 22/30
219/219 ━━━━━━━━━━━━━━━━━━━━ 2s 10ms/step - loss: 0.0745 - val_loss:
0.0745
Epoch 23/30
219/219 ━━━━━━━━━━━━━━━━━━━━ 2s 10ms/step - loss: 0.0744 - val_loss:
0.0744
Epoch 24/30
219/219 ━━━━━━━━━━━━━━━━━━━━ 2s 10ms/step - loss: 0.0743 - val_loss:
0.0743
Epoch 25/30
219/219 ━━━━━━━━━━━━━━━━━━━━ 2s 10ms/step - loss: 0.0742 - val_loss:
0.0743
Epoch 26/30
219/219 ━━━━━━━━━━━━━━━━━━━━ 3s 16ms/step - loss: 0.0742 - val_loss:
```

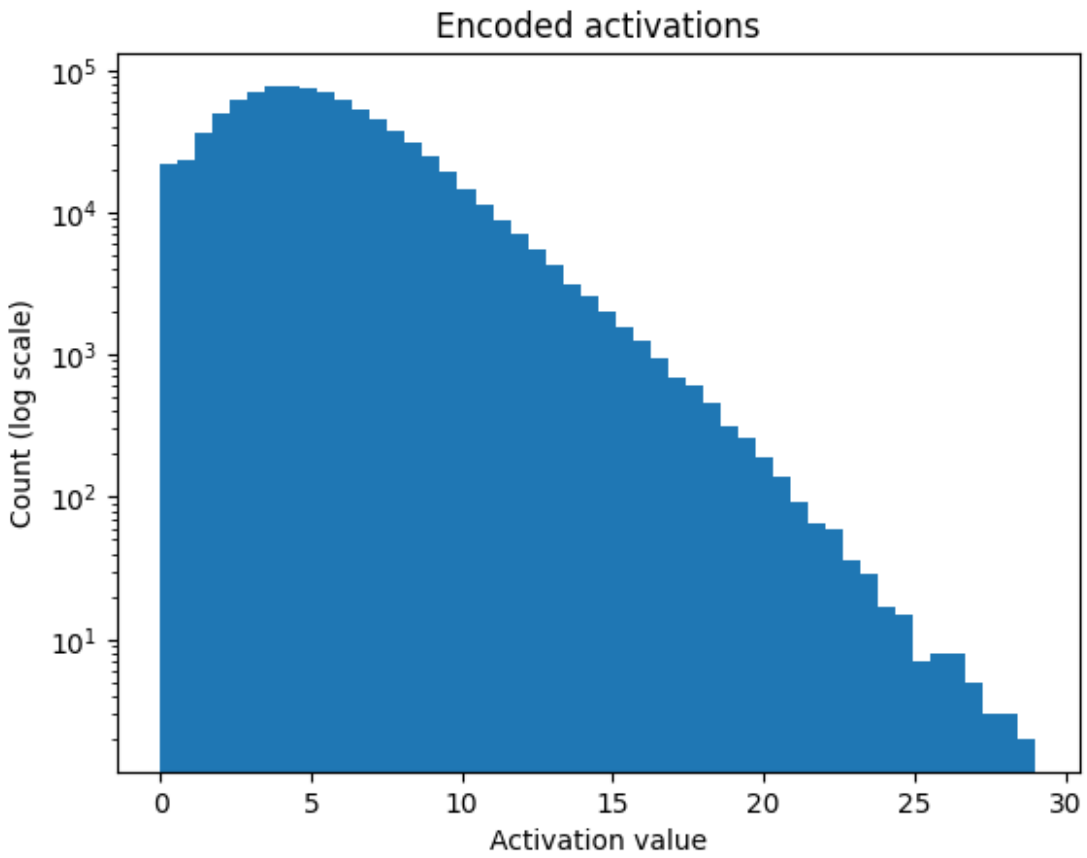
```
0.0742
Epoch 27/30
219/219 ━━━━━━━━━━━━━━━━━━━━ 3s 12ms/step - loss: 0.0741 - val_loss:
0.0742
Epoch 28/30
219/219 ━━━━━━━━━━━━━━━━━━━━ 2s 10ms/step - loss: 0.0741 - val_loss:
0.0741
Epoch 29/30
219/219 ━━━━━━━━━━━━━━━━━━━━ 2s 10ms/step - loss: 0.0740 - val_loss:
0.0741
Epoch 30/30
219/219 ━━━━━━━━━━━━━━━━━━━━ 2s 10ms/step - loss: 0.0740 - val_loss:
0.0740

reconstructed = autoencoder.predict(x_test)
encoded_test = encoder.predict(x_test)

print("Reconstruction MSE:", np.mean((x_test - reconstructed) ** 2))
print("Mean activation in the encoded layer:", encoded_test.mean())
print("Fraction of activations that are exactly zero:", (encoded_test
== 0).mean())
print("Active units per image (average):", (encoded_test >
0).sum(axis=1).mean(), "out of", ENCODING_DIM)

plt.hist(encoded_test.ravel(), bins=50)
plt.yscale("log")
plt.xlabel("Activation value")
plt.ylabel("Count (log scale)")
plt.title("Encoded activations")
plt.show()

438/438 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step
438/438 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step
Reconstruction MSE: 0.0038797886
Mean activation in the encoded layer: 5.293817
Fraction of activations that are exactly zero: 0.009984375
Active units per image (average): 63.361 out of 64
```




```
n = 10
plt.figure(figsize=(15, 3))
for i in range(n):
    plt.subplot(2, n, i + 1)
    plt.imshow(x_test[i].reshape(28, 28), cmap="gray")
    plt.axis("off")
    if i == 0:
        plt.title("Original", loc="left")
    plt.subplot(2, n, n + i + 1)
    plt.imshow(reconstructed[i].reshape(28, 28), cmap="gray")
    plt.axis("off")
    if i == 0:
        plt.title("Reconstructed", loc="left")
plt.show()
```



```
plt.plot(history.history["loss"], label="train loss")
plt.plot(history.history["val_loss"], label="val loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.show()
```

